



TDS6213

Data Structures and Algorithms

Trimester October 2024

Star Gym Membership System

Group 3

Lab Section: 1E

Project Prepared by:

	Name	Student ID	Major
1	Kuan Ing Shien	1231302893	AI
2	Fernando Lim Jia Yee	1211110648	AI
3	Wong Chze Yee	1231303074	AI
4	Dennis Chew Yu Heng	1231303567	AI

**FACULTY OF INFORMATION SCIENCE &
TECHNOLOGY**

MULTIMEDIA UNIVERSITY

MALAYSIA

Gantt Chart

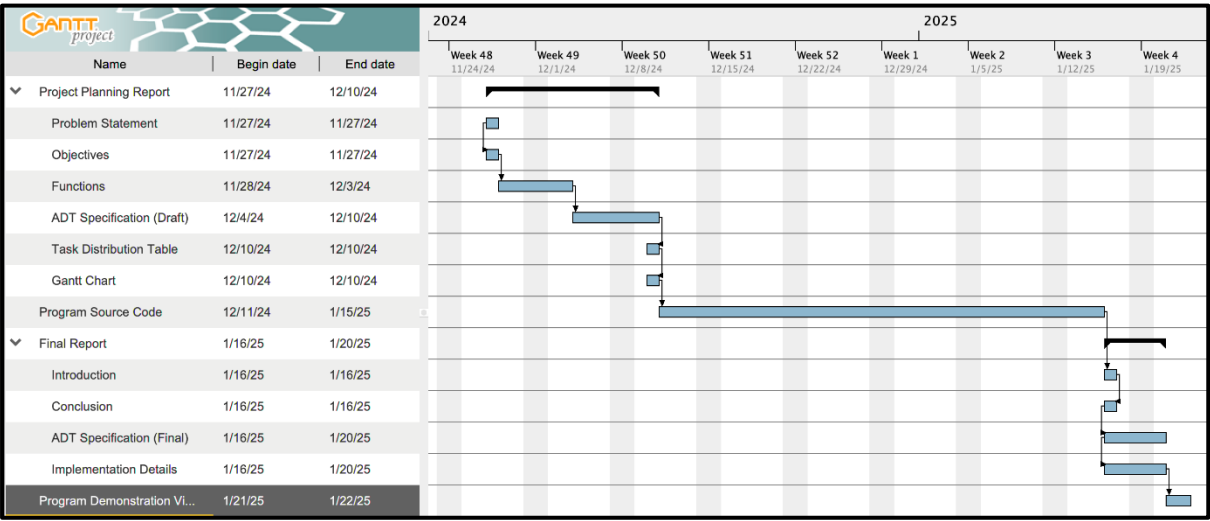


Table of Contents

1.0 Introduction.....	4
2.0 Problem Statements	5
3.0 Objectives	6
4.0 ADT Specifications.....	7
5.0 Implementation Details.....	23
6.0 Program Screenshots.....	28
Worker Login Page	28
Sorting Page	29
User Selection Page	30
Sign Up Page.....	31
Login Page	31
Menu Page	32
Display Profile Page	32
Payment Page.....	33
Equipment Borrow Page	34
Borrow Equipment.....	34
Pending Returns Member List	35
Number of User.....	36
Total Amount Of Equipment Borrowed	37
Return Equipment	38
Class Booking Page	38
Join Class	39
Display Class	39
Number of Students in a Particular Class	40
Total students who registered for class	40
Does the user exist in class	41
Cancel Class.....	41
BMI Calculator Page.....	42
Milestone Page.....	44
7.0 Conclusion	46
8.0 Task Declaration Form	47

1.0 Introduction

For this project, we have created a C++ program which is a fitness management system. Different data structures are used to create this particular program, which include linked-list and hash table, and bubble sort is the sorting algorithm we have used for this system. While creating this program, we did a lot of beta testing and bug fixes to ensure the system runs efficiently while being user friendly.

This project aims to develop a Fitness Management System that allows users to manage their fitness activities, such as checking their profiles, making payments, borrowing equipment, booking classes, calculating BMI, and updating milestones. The system features both a **user interface** for members and an **admin interface** for managing member details. The core functionality of the system revolves around user registration, login, and access to various fitness-related services, all while maintaining security and efficiency in handling user data.

The system is structured into two primary sections: **Admin** and **User**. When users choose the **User** option, they are given the ability to either **sign up** or **log in**. The **Login** process requires users to input their member ID and password. If the entered credentials are valid, users are granted access to a menu of features, such as viewing their profile, making payments, and interacting with other fitness services. The **Sign-Up** process allows new users to create an account by entering their name, password, and receiving a unique member ID generated by the system. The system ensures that duplicate IDs are avoided, enhancing the uniqueness of each member's profile.

On the administrative side, the system provides an interface for managing worker login. The admin can access a separate set of functionalities that are responsible for overseeing the management of users and their data. The login procedure for admins ensures that only authorized personnel can access sensitive parts of the system. Additionally, robust error handling and input validation are incorporated throughout the system to ensure a smooth user experience.

In conclusion, this project seeks to combine essential features of a fitness management system with secure and user-friendly login and registration functionalities. By incorporating both user and admin modules, the system is designed to provide a comprehensive solution for individuals looking to track and manage their fitness goals. The use of queues to store user data, along with validation checks, ensures that the system operates efficiently, providing a seamless experience for users and administrators alike.

2.0 Problem Statements

We have developed a fitness membership system to address several key challenges faced by fitness centers and their members. These challenges include membership management, class scheduling, fitness goal tracking, billing processes, and user engagement.

One of the main problems is the lack of an efficient membership management system. Manual or outdated methods make it difficult to track memberships, process renewals, and ensure smooth user onboarding. This inefficiency leads to missed opportunities for member retention and personalized engagement.

Another significant issue is class scheduling and management. Members often face frustrations due to overbooked classes, lack of real-time information on class availability, and cumbersome booking and cancellation processes. This results in underutilized resources and poor user experience.

Fitness goal tracking and milestone achievements are critical motivators for members, yet fitness centers often lack structured tools to support these needs. Without clear systems for setting targets, monitoring progress, and celebrating achievements (e.g., BMI tracking and milestone updates), members may lose motivation and disengage from their fitness journey.

The billing process also presents challenges. Inconsistent or manual billing systems can result in delays, inaccuracies, and a lack of transparency, causing misunderstandings and disputes between members and management.

Lastly, the absence of mechanisms for personalized health recommendations and feedback collection is a missed opportunity. Features like BMI tracking, health advice, and milestone updates not only enhance user experience but also provide valuable data for improving services and meeting members' evolving needs. By addressing these issues, our fitness membership system ensures a seamless, efficient, and engaging experience for both members and fitness centers.

3.0 Objectives

1. Streamlined Membership Management:

To create a seamless system for users to log in, sign up, renew memberships, and check their statuses efficiently, reducing administrative overhead and enhancing user experience.

2. Enhanced Class Scheduling:

To enable real-time booking, cancellation, and management of class schedules, ensuring optimal utilization of resources and avoiding overbooking or missed sessions.

3. Goal Setting and Progress Tracking:

To provide members with tools to set fitness goals, track their progress, and celebrate milestones, keeping them motivated and engaged in their fitness journey.

4. Transparent Billing and Payment System:

To ensure accurate and timely billing, payment processing, and access to payment histories, creating trust and transparency between members and management.

5. BMI Tracking and Milestone Updates

To incorporate a BMI tracking system that allows members to monitor their health metrics and update milestones, ensuring a more personalized and engaging fitness experience.

4.0 ADT Specifications

1. Login_Or_SignUp ADT

ADT Name: class LOGIN_SIGNUP

Data Items:

- string choice1: Determines if the user is logging in or signing up.
- int memberID: Stores the unique ID of each member.
- string strmemberID: Temporary storage for member ID input.
- string password: Holds the password for authentication.
- string name: Stores the name of the member during sign-up.
- int i: Index for member data.

Operations:

Determine Process

- Determines if the user is logging in or signing up.
- Requirements: c1 must be "1" (login) or "2" (signup).
- Results: Directs the user to the login or sign up process.

Login Process

- Handles the login process for members.
- Requirements: Valid memberID and password.
- Results: Authenticates the user and grants access to the system.

Sign Up process

- Handles the sign-up process for new members.
- Requirements: Valid name and password.
- Results: Creates a new member account and assigns a unique member ID.

2. Worker (Admin) ADT

ADT Name: class WORKER

Data Items:

- string WorkerID: Stores the unique ID of the worker (admin).
- string password: Holds the password for authentication.

Operations:**login()**

- Handles the login process for workers (admins).
- Requirements: Valid WorkerID and password.
- Results: Authenticates the worker and grants access to admin functionalities.

selection()

- Allows workers to sort and view member data (e.g., ID, height, weight, BMI, etc.).
- Results: Displays sorted member data based on the selected criteria.

3. FitnessCentre (Member) ADT**ADT Name: class FitnessCentre****Data Items:**

- int index: Index for member data.
- int id: Stores the unique ID of the member.
- string name: Stores the name of the member.
- string password: Holds the password for authentication.
- bool paymentStatus: Indicates whether the member has paid their fees.
- float height: Stores the member's height.
- float weight: Stores the member's weight.
- float bmi: Stores the member's BMI.
- float targetBmi: Stores the member's target BMI.
- float progression: Tracks the member's progress toward their BMI goal.
- char YorN: Tracks user choices (e.g., "Y" for yes, "N" for no).

Operations:**DisplayProfile(int i)**

- Displays the member's profile information.
- Results: Shows the member's ID, name, payment status, height, weight, BMI, target BMI, and progression.

MakePayment(int i)

- Handles payment for membership fees.
- Results: Updates the member's payment status.

CalculateBMI(int i)

- Calculates the member's BMI and provides health recommendations.
- Results: Displays the member's BMI and health suggestions.

UpdateMilestone(int i)

- Updates the member's progress toward their BMI goal.
- Results: Displays the member's progression and updates their BMI, height, and weight.

BorrowEquipment(int i)

- Allows members to borrow or return gym equipment.
- Results: Updates the equipment borrowing status.

BookClass(int i)

- Allows members to book or cancel fitness classes.
- Results: Updates the class booking status.

4. Class Booking ADT

ADT Name: void BookClass(int i)

Data Items:

- Index: Get the number of index for respective category
- int id: Stores the unique ID of the member.
- int day: Stores the day of the week for the class (1-7).
- string DAY[7]: List of days of the week.
- string strday: input string for user to check if day is numeric
- string choice2: Input the choice for user
- Seperate_Chaining <int> * CLASS_BOOKED=new Seperate_Chaining(): A seperate chaining hash table used for storing id for days booked

Operations:

Add data

- Add a class booking for a member.
- Results: Updates the class booking list.

Delete data

- Cancels a class booking for a member.
- Results: Updates the class booking list.

Display data

- Displays all class bookings.
- Results: Shows the list of booked classes.

Num of users

- Returns the number of users in a specific class.
- Results: Displays the number of participants in a class.

Total amount

- Returns the total number of users registered in all classes.
- Results: Displays the total number of class participants.

Does exist

- Checks if a member exists in a specific class.

- Results: Returns true if the member is in the class, otherwise false.

5. Equipment Borrowing ADT

ADT Name: void BorrowEquipment(int i)

Data Items:

- Int index: Get the number of index for respective category
- int id: Stores the unique ID of the member.
- int equip: Stores the equipment ID.
- string equipment_list[7]: List of available equipment.
- int equipment_list_quantity[7]: Quantity of each equipment.
- int equipment_ID[7]: Equipment IDs.
- string strequip: string representative for equip to validate if it is an integer
- string choice2: Input the choice for user
- Seperate_Chaining* EQUIPMENT=new Seperate_Chaining() : A seperate chaining hash table used for storing id that borrowed the equipment

Operations:

Add data

- Adds equipment borrowing for a member.
- Results: Updates the equipment borrowing list.

Delete data

- Returns borrowed equipment.
- Results: Updates the equipment borrowing list

Display data

- Displays all borrowed equipment.
- Results: Shows the list of borrowed equipment.

Num of users

- Returns the number of users borrowing a specific equipment.
- Results: Displays the popularity of the equipment.

Total amount

- Returns the total number of equipment borrowed.
- Results: Displays the total number of equipment borrowers.

Does exist

- Checks if a member has borrowed a specific equipment.
- Results: Returns true if the member has borrowed the equipment, otherwise false.

6. Payment ADT

ADT Name: void MakePayment(int i)

Data Items:

- Int index: Get the number of index for respective category
- bool paymentStatus: Indicates whether the member has paid their fees.
- int fee: Stores the membership fee.
- int amount: Stores the payment amount.
- string stramount: string representative for amount to validate if it is a float
- Queue<bool>* PAYMENT_STATUS=new Queue<bool>(): A separate queue use to store the status of payment in boolean

Operations:

Insert Item Back

- Updates the payment status for a member.
- Results: Sets the payment status to true or false.

Get Member ID

- Retrieves the payment status for a member.
- Results: Returns the payment status.

7. Milestone ADT

ADT Name: void UpdateMilestone(int i)

Data Items:

- Int index: Get the number of index for respective category
- String temp_target_bmi : string representation for temp_target_bmi to validate is it a float
- float temp_bmi: Stores the member's target BMI.
- String strtemp_height: Indicate if the entered height is a valid number
- String strtemp_weight: Indicate if the entered weight is a valid number
- float temp_height: Stores the member's updated height.
- float temp_weight: Stores the member's updated weight.
- float temp_bmi: Stores the member's pervious BMI.
- float progression: Tracks the member's progress toward their BMI goal.

Operations:

- Display the data of the user.

Insert Item Back

- Updates the progression toward the BMI goal.
- Results: Sets the progression value.

Get Member ID

- Retrieves the progression for a member.
- Results: Returns the progression value.

8. BMI ADT

ADT Name: CalculateBMI(int i)

Data Items:

- Int index: Get the number of index for respective category
- String strtemp_height: Indicate if the entered height is a valid number

- String strcmp_weight: Indicate if the entered weight is a valid number
- float height: Stores the member's height.
- float weight: Stores the member's weight.
- float bmi: Stores the member's BMI.
- Queue<float>* HEIGHT=new Queue<float>(): A separate queue use to store the height of members
- Queue<float>* WEIGHT=new Queue<float>(): A separate queue use to store the weight of members
- Queue<float>* BMI=new Queue<float>(): A separate queue use to store the BMI of members

Operations:

Insert Item Back

- Updates the BMI for a member.
- Results: Sets the BMI value.

Get Member ID

- Retrieves the BMI for a member.
- Results: Returns the BMI value.

Void BUBBLESORT(linked_list<T>* head, string input2)

Data item:

- isSwapped : Indicate if there if no swap in the operation. If there is no swap within the loop, terminate the program.
- linked_list<T>* current: A pointer to track the position in the queue

Operations:

- Sorts member data in ascending or descending order.
- Requirements: input2 must be "1" (ascending) or "2" (descending).
- Results: Sorts and displays the member data.

Formatting ADT

int setNum()

Data item:

- Int num: to store initialize number and return the value

Validating ADT

bool check(string str)

Data item:

- Int num: count the number of decimal point in the string
- Char c: represent each character in the string

Operation:

- Check if the string is a valid integer or float

Formatting ADT

void SetColor(int color)

Operation:

- Set the text color

Login ADT

ADT name: void login()

Data item:

- int chance: Indicates the number of times try to login
- string input1: Indicate the worker ID
- string input2: Indicate the worker's password

Operation:

- Direct user to enter the program or exit the page
- Validate the user

Sorting ADT

ADT name: void login()

Data item:

- int size1: Identifies number of time for looping
- string input1: Direct user to exit or continue the operation
- string input2: Determine if Ascending or Descending operation should be made
- String choice3: Determine which queue for the operation should be applied
- linked_list <int> * ID_temp: Temporary link list for storing member ID for sorting
- linked_list <int> * tail_temp: Temporary link list for indicating queue last element for integer
- linked_list <float> * HEIGHT_temp: Temporary link list for storing member height for sorting
- linked_list <float> * tail_temp: Temporary link list for indicating queue last element for float
- linked_list <float> * WEIGHT_temp: Temporary link list for storing member weight for sorting
- linked_list <float> * BMI_temp: Temporary link list for storing member BMI for sorting
- linked_list <float> * TARGET_BMI_temp: Temporary link list for storing member targeted BMI for sorting
- linked_list <float> * PROGRESSION_temp: Temporary link list for storing member progression for sorting
- linked_list<int>* newNode: Temporary link list for copying queue next element for integer

- `linked_list<float>* newNode`: Temporary link list for indicating link list last element for float

Operation:

- Copy data from the original queue
- Sort data based on user requirement

Formatting ADT

ADT name: `template <typename T> struct linked_list`

Data item:

- `T data`: Indicate data stored in the linked list
- `linked_list* next`: Indicate point to next item

Operation:

- Prepare a link list

Formatting ADT

ADT name: `template <typename T> class Queue`

Data item:

- `linked_list<T>* head`: Indicate start of queue
- `linked_list<T>* tail`: Indicate end of queue
- `int num`: Indicate number of users

Operation:

- `void EnQueue(T data)`
- `void DeQueue()`
- `int Size()`
- `T getMemberID(int i)`
- `int getIDPosition(T Data)`

- void InsertItemBack(T Data, int i)
- bool IsEmpty()

Formatting ADT

ADT name: template <typename T>class Seperate_Chaining

Data item:

- static const int SIZE: Set the size of hash table
- struct HashTable: Prepare a hash table
- HashTable hashTable: Create an object for hashtable
- static int COUNT: Count the number of users

Operation:

- void add_data(T id, int day)
- void delete_data(T id, int day)
- num_of_users_in_array(int day)
- total_amount()
- display_data()
- does_exist(T id, int day)

Formatting ADT

ADT name: struct HashTable

Data item:

- linked_list<T>* table[SIZE]: Construct a hash table for operation

Operation:

- Prepare the hash table

Formatting ADT

ADT name: `template <typename T> void EnQueue(T data)`

Data item:

- `linked_list<T>*` temp: To store a new data item

Operation:

- Store data item from back

Formatting ADT

ADT name: `void DeQueue()`

Data item:

- `linked_list<T>*` temp: To store the first data item

Operation:

- Delete data from front

Formatting ADT

ADT name:

Operation:

- Return number of user

Formatting ADT

ADT name: `template <typename T> T getMemberID(int i)`

Data item:

- T tempID: Store temporary data item for the type
- linked_list<T>* tempLL: Pointer that points through the queue

Operation:

- Provide item that stored in the position I (start from 0)

Formatting ADT

ADT name: `template <typename T> int getIDPosition(T Data)`

Data item:

- int position: Show the position of the item
- linked_list<T>* tempLL: Pointer that points through the queue

Operation:

- Provide the position for the given data
- Provide -1 if the data is not found

Formatting ADT

ADT name: `template <typename T> void InsertItemBack(T Data, int i)`

Data item:

- linked_list<T>* tempLL: Pointer that points through the queue

Operation:

- Find the position given by the user and update with the value given by user

Formatting ADT**ADT name: bool IsEmpty()****Operation:**

- Check if the queue is empty

Formatting ADT**ADT name: template <typename T> void add_data(T id, int day)****Data item:**

- linked_list<T>* temp: To check if the link list in the table is empty
- linked_list<T>* temp1: Act as a link list to store data and attached to the table

Operation:

- Check if the table has no link list attached to it
- Attached link list from the front (Similar to stack)

Formatting ADT**ADT name: template <typename T> void delete_data(T id, int day)****Data item:**

- linked_list<T>* temp: Loop through the link list for a hash table
- linked_list<T>* pre: Use to track the previous link list in the hash table

Operation:

- Delete link list from the front (Similar to stack)

Formatting ADT**ADT name: void display_data()**

Operation:

- Display data throughout the table

Formatting ADT

ADT name: int num_of_users_in_array(int day)

Data item:

- int Num_Users: count number of users in the link list for a specific part of hash table
- linked_list<T>* temp: Loop through the link list for a hash table

Operation:

- Loop through a specific part of hash table and count the number of link list

Formatting ADT

ADT name: int total_amount()

Operation:

- Return the total amount of user

Formatting ADT

ADT name: template <typename T> bool does_exist(T id, int day)

Data item:

- linked_list<T>* temp: Loop through the link list for a hash table

Operation:

- Check if the data given is found on specific part of the table

5.0 Implementation Details

Data Structures

1. Linked List

The linked list is implemented as a templated structure to store various types of data, such as member information and hash table entries. It forms the backbone for the Queue and Hash Table (Separate Chaining) implementations. Each node in the linked list contains data and a pointer to the next node, allowing dynamic memory allocation.

Advantages of using Linked Lists:

A linked list is dynamic, so it efficiently manages memory without requiring predefined sizes, unlike arrays. This flexibility is crucial for operations where the data size is unknown or can change frequently, as in a fitness system where members, equipment, or bookings can vary over time.

Reason of choosing Linked List:

The fitness system manages various entities like member IDs, equipment bookings, and class schedules, which are dynamic in nature. The linked list allows these data structures to expand or contract as needed without wasted memory or resizing overhead.

2. Hash Table with Separate Chaining

The `Seperate_Chaining` class is a hash table implementation that uses linked lists for collision resolution. This is employed for tasks like booking classes or tracking equipment borrowing.

Advantages of using Hash Table:

The hash table provides average constant-time complexity for insertions and lookups, while separate chaining ensures that collisions (multiple keys hashing to the same index) are efficiently resolved by maintaining a linked list for each hash bucket.

Reason of choosing Hash Table over stack:

The fitness system involves managing diverse entities like class bookings and equipment loans. A hash table with separate chaining ensures quick access while gracefully handling collisions when multiple users borrow equipment or book the same class.

3. Queue

The Queue class is implemented using a linked list to manage member attributes like IDs, names, passwords, payment statuses, and progressions. It supports typical queue operations like enqueue (`EnQueue`), dequeue (`DeQueue`), and checking if the queue is empty (`IsEmpty`).

Advantages of using Queue:

Queues provide first-in-first-out (FIFO) ordering, ensuring that data is processed in the same sequence it was added. Using a linked list as the underlying structure allows dynamic sizing and efficient memory utilization.

Reason of choosing Queue:

Queues are ideal for managing sequential operations, such as processing members' information or class schedules, where the order of operations is crucial.

4. Static Arrays

Static arrays are used to store predefined data like equipment names, quantities, and weekdays. For instance, the `equipment_list`, `equipment_list_quantity`, and `DAY` arrays are fixed-size arrays.

Advantages of using Static Arrays:

Static arrays are simple and efficient for managing fixed-size data that doesn't require dynamic resizing. Access time is constant, making them ideal for lookups.

Reason to use Static Arrays:

Predefined data, such as equipment names or weekdays, doesn't change in size. Using static arrays simplifies the implementation and provides efficient access to this data.

5. Worker and FitnessCentre Classes

`WORKER` class handles admin-specific functions like login and class member management while `FitnessCentre` class manages operations for individual members, including displaying profiles, calculating BMI, booking classes, and borrowing equipment.

Advantages:

Object-oriented design promotes modularity, reusability, and clarity. Each class encapsulates its responsibilities, making the system easier to maintain and extend.

Reason for Use:

The fitness system involves distinct roles (admin and users) and modular operations. Classes organize these functionalities logically, improving readability and scalability.

Why do we choose these data structures over stack for our program?

The data structures used in the system—linked lists, queues, hash tables, and static arrays—were chosen over stacks because they better align with the system's requirements for flexibility, accessibility, and dynamic data management. While stacks operate on a strict last-in, first-out (LIFO) principle, the system often requires random access, sequential

processing (FIFO), and efficient searching or collision handling, which stacks cannot provide. For instance, queues ensure orderly processing of member data, hash tables provide fast lookups and collision resolution, and linked lists enable dynamic resizing. Stacks, with their restrictive access pattern, are less suited for these diverse and interactive workflows.

Sorting Algorithm

- Bubble Sort

Bubble sort is used to arrange data (e.g., member IDs, heights, weights, BMI) in ascending or descending order based on user input. The algorithm iteratively compares adjacent elements and swaps them if they are out of order.

Advantages of using Bubble Sort:

Bubble sort is simple and easy to implement. For small datasets, such as sorting a few member attributes in this fitness system, it performs adequately. Its adaptive nature can also optimize sorting for nearly sorted datasets.

Reason to use Bubble Sort:

Sorting operations within the fitness system occur infrequently and on small datasets, which bubble sort's simplicity outweighs its inefficiency for larger datasets. It also ensures a clear and maintainable implementation for basic sorting needs.

Why do we chose bubble sort over other sorting algorithm?

Bubble sort was chosen because of its simplicity and adequacy for the fitness system's small datasets and infrequent sorting operations. While more efficient algorithms like merge sort or quicksort offer better performance for large datasets, their complexity is unnecessary for sorting tasks such as organizing a few member attributes or fitness progression stats. Bubble sort's straightforward implementation and in-place sorting mechanism make it easy to understand, maintain, and integrate into the system, outweighing its inefficiencies for the system's modest data sizes.

Search Algorithm

1. Linear Search (Queue)

The queue uses a linear search to find the position of a specific data item (e.g., member ID or equipment availability). This is achieved by traversing the linked list sequentially.

Advantages from using this:

Linear search is straightforward to implement and works well for small datasets, as its performance is unaffected by how the data is stored or the lack of additional structures like indexes.

Reason to use this:

Since the fitness system queues are small and the search operations are infrequent, the linear search provides a simple and direct solution without introducing additional complexity or memory overhead.

2. Linear Search (Hash Table with Separate Chaining)

The `does_exist` and `delete_data` functions in the `Seperate_Chaining` class use linear search within linked lists to locate a specific item in a hash table bucket.

Advantages from using this:

Linear search complements separate chaining by efficiently searching the small linked lists in each hash table bucket. It ensures that collision resolution doesn't degrade the system's functionality.

Reason to use this:

The hash table resolves collisions by storing multiple items in linked lists. These lists are typically short, making linear search efficient and appropriate in this context.

Why do we use linear search instead of other search algorithms?

Linear search is used because it is simple and effective for the system's small, unsorted datasets. Unlike binary search or hash-based searching, which require sorted data or additional preprocessing, linear search works directly on the dynamically changing data stored in queues and linked lists. This straightforward approach is sufficient for the fitness system, where search operations are infrequent and datasets are small, making the overhead of more complex algorithms unnecessary.

Additional Algorithms**1. Password Validation and Error Handling**

The `check` function verifies if user inputs, such as numbers or floats, are valid. Passwords are also validated during login.

Advantages:

Input validation ensures system robustness by preventing invalid or harmful inputs from disrupting the flow.

Reason for Use:

Given the reliance on user inputs, input validation ensures data integrity and protects against potential errors or misuse.

2. Console Colour Coding

Colour codes (e.g., RED_BG, GREEN_BG) provide visual cues for BMI ranges and other statuses.

Advantages:

Visual feedback enhances user experience by quickly conveying information (e.g., underweight, normal, overweight).

Reason for Use:

In a fitness application, color coding improves engagement and clarity, helping users understand their progress and status at a glance.

3. User Input and Selection Loops

The system uses loops to repeatedly prompt users for inputs until valid data is provided or the desired operation is completed.

Advantages:

This ensures that users don't accidentally exit critical workflows or proceed with incomplete or invalid inputs.

Reason for Use:

The system involves interactive workflows, such as class bookings or equipment loans, where user inputs are vital. Loops maintain usability and guide users toward completing their tasks.

6.0 Program Screenshots

Login Selection Page

```
=====
SSSSS  TTTT  AAAA  RRRR      GGG  Y  Y  M  M
S      T    A    A R  R      G    Y Y  MM MM
SSS    T    AAAAAA RRRRR      G  GG  Y  M M M
S      T    A    A R  R      G  G  Y  M  M
SSSSS  T    A    A R  R      GGG  Y  M  M
=====
Are you an admin or user?
1. Admin
2. User
|
```

This is the **Login Selection Page** of the **Fitness Membership System**. It allows users to select their role by choosing between **Admin** or **User**, determining the functionality they can access.

Worker Login Page

```
=====
SSSSS  TTTT  AAAA  RRRR      GGG  Y  Y  M  M
S      T    A    A R  R      G    Y Y  MM MM
SSS    T    AAAAAA RRRRR      G  GG  Y  M M M
S      T    A    A R  R      G  G  Y  M  M
SSSSS  T    A    A R  R      GGG  Y  M  M
=====
Are you an admin or user?
1. Admin
2. User
1|
```

```
=====
Worker Login Page!
=====
If both input and output are 0, return.
Please enter your WorkerID :|
```

```
=====
Worker Login Page!
=====
If both input and output are 0, return.
Please enter your WorkerID :Worker666
Please enter your password :ILikeToWork|
```

If we choose the first option, the program will lead us to the Admin login page and you need to insert the WorkerID: “Worker666” and password: “ILikeToWork”. It will lead us to the Admin Page.

```
=====
      Worker Login Page!
=====
If both input and output are 0, return.
Please enter your WorkerID :Worker555
Please enter your password :IHateToWork
Invalid credentials. Attempts remaining: 2
|
```

If the credentials are invalid, the system displays the remaining attempts and loops to prompt for the correct ID and password again.

Sorting Page

```
=====
      Sorting Page!
=====
Which domain do you want to search :
1. Members
0. Return
|
```

Once we login to the worker account, the program will lead us to **Sorting Page**. The **Sorting Page** allows administrators to choose between searching **Members** or **Return** to the previous screen. Selecting **1 (Members)** proceeds to sorting options for member data.

```
=====
      Sorting Page!
=====
Which domain do you want to search :
1. Members
0. Return
1

Ascending or Descending
1. Ascending
2. Descending
```

After selecting **Members** on the Sorting Page, the worker is prompted to choose between **Ascending** or **Descending** order. This determines the direction in which member data will be sorted and determining how the information will be displayed.

```

=====
                Sorting Page!
=====
Which domain do you want to search :
1. Members
0. Return
1

Ascending or Descending
1. Ascending
2. Descending
1

Which selection do you want to sort :
1. Member ID
2. Height
3. Weight
4. BMI
5. Target BMI
6. Progression

```

Once the sorting order is selected, they can further specify which member attribute to sort by, such as **Member ID**, **Height**, **Weight**, **BMI**, **Target BMI**, or **Progression**.

User Selection Page

```

Are you an admin or user?
1. Admin
2. User
2|

```

If we choose **2 (User)** , the program will lead us to **User Selection Page**.

```

=====
    User Selection - Fitness System
=====
Welcome to Fitness System, please input a selection with respective number
1. Login
2. Sign Up
|

```

The **User Selection Page** welcomes users to the Fitness System and provides two options: **Login** for existing members or **Sign Up** for new members to create an account. This choice determines the next steps in the user flow.

Sign Up Page

```
=====
User Selection - Fitness System
=====
Welcome to Fitness System, please input a selection with respective number
1. Login
2. Sign Up
2|

Please enter your name:
Wong
Please enter your password:
1234|
```

After selecting **Sign Up**, the user is prompted to enter their **name** and **password** as the steps in creating a new account to complete the registration process.

```
Please enter your name:
Wong
Please enter your password:
1234
This is your member ID : 13443
Congratulations! Press any key to exit.|
```

Once both fields are filled, the system generates a member ID for new user. A confirmation message, "**Congratulations! Press any key to exit,**" is displayed, indicating successful account creation.

Login Page

```
=====
User Selection - Fitness System
=====
Welcome to Fitness System, please input a selection with respective number
1. Login
2. Sign Up
1|
```

```
=====
Welcome to Login Page!
=====
Press -1 for member ID to return
Please enter your memberID and password
member ID: |
```

After creating an account, users can select **Login** to proceed to the **Login Page**. Here, existing members are prompted to enter their **member ID** and **password** to access their account. If the user enters **-1** for the member ID, they are returned to the previous screen.

Menu Page

```
=====
Welcome to Menu Page!
=====
Which option do you want to choose?
0. Return
1. Check profile
2. Make payment
3. Borrow equipment
4. Book class
5. Calculate BMI
6. Update Milestone
```

After successfully logging in, users are directed to the **Menu Page**, which offers various options: **Check Profile**, **Make Payment**, **Borrow Equipment**, **Book Class**, **Calculate BMI**, **Update Milestone**, or **Return** to the previous screen. This page serves as the main hub for accessing fitness system features.

Display Profile Page

```
=====
Welcome to Menu Page!
=====
Which option do you want to choose?
0. Return
1. Check profile
2. Make payment
3. Borrow equipment
4. Book class
5. Calculate BMI
6. Update Milestone
1|
```



```
=====
Display Profile
=====
ID      :14246
Name    :Wong
Status  :0
Height  :0.000000
Weight  :0.000000
BMI     :0.000000
Target Bmi :0.000000
Progression :0.000000
Press any key to exit|
```

After selecting 1 (**Check Profile**), the system displays the **Display Profile**, showing the user's detailed information, including **ID**, **Name**, **Status**, **Height**, **Weight**, **BMI**, **Target BMI**, and **Progression**. Once the user reviews their profile, they can press any key to exit and return to the Menu Page.

Payment Page

```
=====
Welcome to Menu Page!
=====
Which option do you want to choose?
0. Return
1. Check profile
2. Make payment
3. Borrow equipment
4. Book class
5. Calculate BMI
6. Update Milestone
2|
```

```
=====
Payment Page
=====
You have not paid your member fee, the fee is RM50. You have owe RM50
Please enter your payment: RM|
```

After selecting **Make Payment**, the **Payment Page** displays the outstanding member fee (e.g., RM50) and prompts the user to enter the payment amount. This allows users to settle their dues and update their payment status.

```
=====
Payment Page
=====
You have not paid your member fee, the fee is RM50. You have owe RM50
Please enter your payment: RM50
This is your balance: RM0
Press any key to exit|
```

After entering the payment amount: RM50, the system calculates the balance: RM0 and confirms the payment. A message, "**Press any key to exit,**" is displayed, allowing the user to return to the Menu Page after completing the transaction.

Equipment Borrow Page

```
=====
Welcome to Menu Page!
=====
Which option do you want to choose?
0. Return
1. Check profile
2. Make payment
3. Borrow equipment
4. Book class
5. Calculate BMI
6. Update Milestone
3|
```

```
=====
Equipment Borrow
=====
Please select a choice with respect to the number:
1. Borrow equipment
2. Return equipment
3. Who is still owe
4. Number of users
5. Total amount of equipment borrowed
0. Return
|
```

After selecting 3 (**Borrow Equipment**), the **Equipment Borrow Page** provides options to **Borrow Equipment**, **Return Equipment**, check **Pending Returns Member List**, view the **Number of Users**, see the **Total Amount of Equipment Borrowed**, or **Return** to the Menu Page. This page facilitates equipment management for users.

Borrow Equipment

```
=====
Equipment Borrow
=====
Please select a choice with respect to the number:
1. Borrow equipment
2. Return equipment
3. Pending Returns Member List
4. Number of users
5. Total amount of equipment borrowed
0. Return
1|
```

```
Which equipment do you want to borrow?
```

1. 5kg Dumbbells
 2. Resistance Bands
 3. Jump Rope
 4. Foam Roller
 5. Medicine Ball
 6. TRX Suspension Trainer
 7. Exercise Ball
- ```
|
```

After selecting **Borrow Equipment**, the **Equipment Borrow Selection Page** lists available equipment options, such as **5kg Dumbbells**, **Resistance Bands**, **Jump Rope**, **Foam Roller**, **Medicine Ball**, **TRX Suspension Trainer**, and **Exercise Ball**. Members can choose the desired equipment to proceed with the borrowing process.

```
Which equipment do you want to borrow?
```

1. 5kg Dumbbells
  2. Resistance Bands
  3. Jump Rope
  4. Foam Roller
  5. Medicine Ball
  6. TRX Suspension Trainer
  7. Exercise Ball
- ```
1
```

If the member selects **5kg Dumbbells**, the system checks availability, processes the request, and updates the equipment list to reflect the **Pending Returns Member List**, **Number of Users** and **Total Amount of equipment borrowed**.

Pending Returns Member List

```
=====
Equipment Borrow
=====
```

```
Please select a choice with respect to the number:
```

1. Borrow equipment
 2. Return equipment
 3. Pending Returns Member List
 4. Number of users
 5. Total amount of equipment borrowed
 0. Return
- ```
3|
```

```
=====
Equipment Borrow
=====
Please select a choice with respect to the number:
1. Borrow equipment
2. Return equipment
3. Pending Returns Member List
4. Number of users
5. Total amount of equipment borrowed
0. Return
3
Selection 1 : 19605
Selection 2 :
Selection 3 :
Selection 4 :
Selection 5 :
Selection 6 :
Selection 7 :
Enter any input to exit the program|
```

After selecting **Pending Returns Member List**, the system displays members who have borrowed equipment, such as member ID **19605** borrowing selection 1 which is **5kg Dumbbells**. Once reviewed, the user can enter any input to exit and return to the Borrow Equipment Page.

### Number of User

```
=====
Equipment Borrow
=====
Please select a choice with respect to the number:
1. Borrow equipment
2. Return equipment
3. Pending Returns Member List
4. Number of users
5. Total amount of equipment borrowed
0. Return
4|

Which item's popularity do you want to check?
1. 5kg Dumbbells
2. Resistance Bands
3. Jump Rope
4. Foam Roller
5. Medicine Ball
6. TRX Suspension Trainer
7. Exercise Ball
|
```

After selecting **Number of Users**, the system prompts the user to choose an item (e.g., **5kg Dumbbells**, **Resistance Bands**, etc.) to check its popularity. This displays how many users have borrowed the selected equipment.

```

Which item's popularity do you want to check?
1. 5kg Dumbbells
2. Resistance Bands
3. Jump Rope
4. Foam Roller
5. Medicine Ball
6. TRX Suspension Trainer
7. Exercise Ball
1
The popularity of 5kg Dumbbells is 1
Enter any input to exit the program|

```

After selecting an item: **5kg Dumbbells**, the system displays its popularity: **1** user has borrowed it. The user can then enter any input to exit and return to the previous page.

### Total Amount Of Equipment Borrowed

```

=====
Equipment Borrow
=====
Please select a choice with respect to the number:
1. Borrow equipment
2. Return equipment
3. Pending Returns Member List
4. Number of users
5. Total amount of equipment borrowed
0. Return
5|

```

```

=====
Equipment Borrow
=====
Please select a choice with respect to the number:
1. Borrow equipment
2. Return equipment
3. Pending Returns Member List
4. Number of users
5. Total amount of equipment borrowed
0. Return
5
Total amount of equipment borrowers are :1
Enter any input to exit the program|

```

After selecting **Total Amount of Equipment Borrowed**, the system displays the total number of equipment borrowers: **1**.

## Return Equipment

```
=====
Equipment Borrow
=====
Please select a choice with respect to the number:
1. Borrow equipment
2. Return equipment
3. Pending Returns Member List
4. Number of users
5. Total amount of equipment borrowed
0. Return
2|

Which equipment do you want to return?
1. 5kg Dumbbells
2. Resistance Bands
3. Jump Rope
4. Foam Roller
5. Medicine Ball
6. TRX Suspension Trainer
7. Exercise Ball
1|
```

After selecting **Return Equipment**, the system prompts the user to choose the equipment they want to return : **5kg Dumbbells**. This initiates the return process for the selected item.

## Class Booking Page

```
=====
Welcome to Menu Page!
=====
Which option do you want to choose?
0. Return
1. Check profile
2. Make payment
3. Borrow equipment
4. Book class
5. Calculate BMI
6. Update Milestone
|
```

```
=====
Class Booking
=====
Please select a choice with respect to the number:
1. Join a Class
2. Cancel a Class
3. Display Class
4. Number of students in a particular class
5. Total students who registered in the class
6. Does user exist in class
0. Return
|
```

Upon selecting **Book Class**, the **Class Booking Page** appears, displaying an interface with options to **Join** a class, **Cancel** a class, **Display Available Classes**, **Number of Students in a Particular Class**, **Total Students who Registered in the class** and **Does User Exist in Class**. The member can navigate through these options to manage their class bookings.

### Join Class

```
Please select a day with respect number you want to add class:
1. Sunday
2. Monday
3. Tuesday
4. Wednesday
5. Thursday
6. Friday
7. Saturday
```

```
Please select a day with respect number you want to add class:
1. Sunday
2. Monday
3. Tuesday
4. Wednesday
5. Thursday
6. Friday
7. Saturday
1|
```

In **Join Class** from the **Class Booking Page**, the system prompts the user to select a **Day** for the class. The available days of the week are displayed, including **Sunday, Monday, Tuesday, Wednesday, Thursday, Friday**, and **Saturday**. The member can choose a day, and upon selection, they can proceed with further options to add a class for that specific day.

### Display Class

```
Selection 1 : 18698
Selection 2 :
Selection 3 :
Selection 4 :
Selection 5 :
Selection 6 :
Selection 7 :
Press any key to exit|
```

After selecting **Display Class** from the **Class Booking Page**, the system displays a list of available days for class bookings, with **Sunday** as the first selection. For each selected day, the member IDs of those who have joined the class are shown. The page also includes an option to **Exit** if the user wishes to leave without making any changes.

#### Number of Students in a Particular Class

```
Please enter your day:
```

1. Sunday
2. Monday
3. Tuesday
4. Wednesday
5. Thursday
6. Friday
7. Saturday

```
|
```

```
Please enter your day:
```

1. Sunday
2. Monday
3. Tuesday
4. Wednesday
5. Thursday
6. Friday
7. Saturday

```
1
```

```
There is 1 students in the class
```

```
Press any key to exit|
```

By selecting **Number of Students in a Particular Class** from the **Class Booking Page**, the system prompts the user to enter a specific day of the week. Once a day is entered, the system displays the number of students currently enrolled in the class for that day. Additionally, there is an option to **Exit** the page without making any changes.

#### Total students who registered for class

```
Total amount of users in the class are :1
```

```
Press any key to exit|
```

After selecting **Total Students Registered for Class** from the **Class Booking Page**, the system displays the total number of users who have registered across all available classes. It also provides an option to **Exit** the page.



### Does the user exist in class

```
Please enter your day:
1. Sunday
2. Monday
3. Tuesday
4. Wednesday
5. Thursday
6. Friday
7. Saturday
1
You exist in the class
Press any key to exit|
```

In **Does the User Exist in Class** from the **Class Booking Page**, the system prompts the user to enter a day of the week. It then confirms whether the user is registered for the class on that day and provides an option to Exit the page.

### Cancel Class

```
Please select a day with respect number that you want to cancel class on:
1. Sunday
2. Monday
3. Tuesday
4. Wednesday
5. Thursday
6. Friday
7. Saturday
1|
```

Upon selecting **Cancel Class** from the **Class Booking Page**, the system prompts the user to select a day of the week for cancelling a class. Options for each day are available, allowing the user to choose the day they wish to cancel.

```
Please enter your day:
1. Sunday
2. Monday
3. Tuesday
4. Wednesday
5. Thursday
6. Friday
7. Saturday
1
There is 0 students in the class
Press any key to exit|
```

After that, the system updates the **Number of Students** in Class page. The user is prompted to enter a day, and the system confirms that no students remain in the class for that day. Then, there is an option to **Exit**.

## BMI Calculator Page

```
=====
BMI Calculator
=====
Please enter your weight (in kg): 44
Please enter your height (in m): 1.53|
```

The **BMI Calculator Page** displays an input screen where the user is prompted to enter **weight and height**. Once both fields are filled in, the user can proceed to calculate their **BMI**. However, the calculation results are not yet shown, awaiting the necessary inputs from the user.

```
=====
BMI Calculator
=====
Please enter your weight (in kg): 44
Please enter your height (in m): 1.53
This is your BMI :18.8
Congretulations! You reached the normal BMI. This is your BMI:18.8
Suggestions:
1. Continue eating a healthy and varied diet to maintain current weight and good health.
2. Engage in regular cardiovascular exercise (like walking, cycling, or swimming) and strength training (2-3 times a week) to maintain muscle mass and cardiovascular health.
3. Prioritize quality sleep (7-9 hours per night) to support metabolism and overall health.

It is hard to persist in normal BMI! Keep the spirit up!
Do you want to re-enter your BMI? [Y/N]
|
```

After entering weight and height, the **BMI Calculator Result Page** displays a **normal BMI** range result. It congratulates the user on maintaining a healthy BMI and provides suggestions for maintaining their health, such as regular exercise and balanced nutrition. An **option to re-enter BMI details** is also available for users who wish to update their information.

```
=====
BMI Calculator
=====
Please enter your weight (in kg): 36
Please enter your height (in m): 1.53
This is your BMI :15.38
You are Severely Underweight! This is your BMI:15.38
Suggestions:
1. Focus on high-calorie, nutrient-rich foods.
2. Eat small, frequent meals (5-6 times a day) to increase calorie intake.
3. Prioritize protein (chicken, fish, eggs, legumes) and healthy fats (olive oil, nut butter, fatty fish) to build muscle mass.
Do you want to re-enter your BMI? [Y/N]
```

```
=====
BMI Calculator
=====
Please enter your weight (in kg): 60
Please enter your height (in m): 1.53
This is your BMI :25.63
You are Overweight! This is your BMI:25.63
Suggestions:
1. Aim for a moderate calorie deficit to promote gradual weight loss. Reduce portion sizes and opt for lower-calorie, nutrient-dense foods.
2. Minimize sugary beverages, snacks, and processed foods.
3. Eat slowly and stop eating when you feel satisfied, not full.
Do you want to re-enter your BMI? [Y/N]
```

```
=====
BMI Calculator
=====
Please enter your weight (in kg): 100
Please enter your height (in m): 1.53
This is your BMI :42.72
You are Severely Obese! This is your BMI:42.72
Suggestions:
1. Seek support from healthcare providers for an individualized weight loss plan.
2. Implement a low-calorie, nutrient-dense diet that prioritizes protein and fiber, while minimizing processed foods and sugars.
3. Engage in low-impact, consistent physical activity, aiming for gradual improvement in fitness and mobility.
Do you want to re-enter your BMI? [Y/N]
```

On the other hand, after entering weight and height, the **BMI Calculator Result Page** displays a result indicating **severe obesity**, **overweight**, or **underweight** status. The user's **BMI** is shown along with **personalized suggestions for improvement**, such as diet changes or increased physical activity. An **option to re-enter BMI details** is also available for users who wish to update their information.

## Milestone Page

```
=====
Update Milestone for your desired BMI
=====
What is your targeted BMI? :21|
```

After selecting Milestone, the **Milestone Page** prompts the user to **input their targeted BMI**. This allows the user to set a **fitness goal** and track their progress towards achieving it.

```
=====
Update Milestone for your desired BMI
=====
What is your weight after progression? (kg):45
What is your height after progression? (m):1.60|
```

The **Milestone Update Page** prompts the user to **input their current weight and height**. This allows the user to track their progress and update their **BMI milestone**.

```
=====
Update Milestone for your desired BMI
=====
What is your weight after progression? (kg):45
What is your height after progression? (m):1.60
This is your progression: -0.554545
From previous height: 1.53
To current height: 1.6
From previous weight: 44
To current weight: 45
From previous BMI: 18.8
To current BMI: 17.58
Press any key to exit|
```

After entering the current height and weight, this image displays the **BMI milestone update** with the user's **weight** and **height** inputs, showing the **BMI progression** (previous and current: height, weight, BMI). Additionally, there is an **option to exit the system**.

```
=====
Update Milestone for your desired BMI
=====
What is your weight after progression? (kg):55
What is your height after progression? (m):1.53
This is your progression: 1.73099
From previous height: 1.6
To current height: 1.53
From previous weight: 45
To current weight: 55
From previous BMI: 17.58
To current BMI: 23.5
Congretulations for finishing your goal!
Do you want to set new goal? [Y/N]
|
```

```
=====
Update Milestone for your desired BMI
=====
What is your weight after progression? (kg):55
What is your height after progression? (m):1.53
This is your progression: 1.73099
From previous height: 1.6
To current height: 1.53
From previous weight: 45
To current weight: 55
From previous BMI: 17.58
To current BMI: 23.5
Congretulations for finishing your goal!
Do you want to set new goal? [Y/N]
n
Thanks for using our system!
Press any key to exit|
```

After reaching the targeted BMI goal, the system congratulates the user for achieving their goal. It prompts the user with the option to **set a new goal**. Users can choose whether to **set a new target or exit the system** with an appreciation message.

## **7.0 Conclusion**

In this project, we can learn about the programming language through the usage of different data structure types, sorting and search algorithms within the program we have created. The fitness membership system that we created is an extremely useful piece of software that can assist gym businesses in managing their customer data. While this program might look complicated on the surface, the actual usage of the fitness membership system is easy and user friendly as it does not require a huge learning curve from the user. The system's design final goal is to ensure efficient management of memberships, class bookings, and member progression, which help reflects thoughtful application of computational concepts.

Our system employs a bunch of data structures, such as linked list which helps the businesses to handle dynamic datasets, such as member profiles, class schedules, and equipment usage efficiently with a lot of flexibility, queue which ensures orderly processing of member data and sequential operations, critical for maintaining system integrity in a multi-user environment and hash table with separate chaining, which supports rapid access to class and equipment data, meeting the needs of a diverse user base.

Other than that, our system employs bubble sort for data organization for sorting small datasets, such as member attributes or progression statistics, where sorting operations are infrequent, and the datasets are manageable in size. And for search operations, linear search is implemented in both queues and hash table linked lists. Its straightforwardness and reliability make it ideal for small, dynamic datasets.

The integration of all these components supports the fitness membership system's goals of efficient membership management, seamless class scheduling, accurate customer billing, and effective fitness tracking. By leveraging modular object-oriented programming principles, the system maintains scalability and maintainability, ensuring that future expansions or modifications can be accommodated easily.

To conclude this project, this program is an effective software to deal with the problems with customer management of a gym business and we hope it can be used to assist any gym businesses in helping to manage their customer data swiftly and efficiently in the simplest and easiest way possible. With all the features the fitness management system provides, we can assure that this program is able to assist the business in customer data management.

## **8.0 Task Declaration Form**

| <b>Task Declaration Form</b>                                                                                                                                                                                                                                                                                                                                  |   |                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|----------------|
| Each group member, including the group leader, must individually complete and sign this form.<br>The signed forms (from all members) are to be appended to the final report for submission.                                                                                                                                                                   |   |                |
| <b>STAR GYM MEMBERSHIP SYSTEM</b>                                                                                                                                                                                                                                                                                                                             |   |                |
| by                                                                                                                                                                                                                                                                                                                                                            |   |                |
| <b>1E_GROUP 3</b>                                                                                                                                                                                                                                                                                                                                             |   |                |
| <b>Group Member's Name</b>                                                                                                                                                                                                                                                                                                                                    | : | Kuan Ing Shien |
| <b>Group Member's ID</b>                                                                                                                                                                                                                                                                                                                                      | : | 1231302893     |
| <p>For the purpose of completing this project, I have performed the following tasks:</p> <ul style="list-style-type: none"><li>• Finalize the report and source code</li><li>• Complete Program Screenshots</li><li>• Finalize ADT Specification</li><li>• Test and check for bugs in program.</li><li>• Distribute and conduct the whole project .</li></ul> |   |                |
| <p>1. I hereby declare that I have assessed the final submission and I take the full responsibility should there be any incompleteness, omissions, delays, or non-submission.</p> <p>2. I also certify that not part of this project is copied from any other student's work, or from any other sources, except where acknowledgement is made.</p>            |   |                |
| <b>Group Member's Signature</b>                                                                                                                                                                                                                                                                                                                               | : | <i>Kuan</i>    |
| <b>Group Member's Name</b>                                                                                                                                                                                                                                                                                                                                    | : | Kuan Ing Shien |
| <b>Group Member's ID</b>                                                                                                                                                                                                                                                                                                                                      | : | 1231302893     |
| <b>Date</b>                                                                                                                                                                                                                                                                                                                                                   | : | 22/1/2025      |

### Task Declaration Form

Each group member, including the group leader, must individually complete and sign this form.  
The signed forms (from all members) are to be appended to the final report for submission.

#### STAR GYM MEMBERSHIP SYSTEM

by

**1E\_GROUP 3**

**Group Member's Name** : Fernando Lim Jia Yee

**Group Member's ID** : 1211110648

For the purpose of completing this project, I have performed the following tasks:

- Program most of the program
- Write the ADT specifications
- Done bug testing
- Cooperate with my teammate to complete the task

1. I hereby declare that I have assessed the final submission and I take the full responsibility should there be any incompleteness, omissions, delays, or non-submission.

2. I also certify that not part of this project is copied from any other student's work, or from any other sources, except where acknowledgement is made.

**Group Member's Signature** :

*Fernando*

**Group Member's Name** :

Fernando Lim Jia Yee

**Group Member's ID** :

1211110648

**Date** :

21/01/2025



### Task Declaration Form

Each group member, including the group leader, must individually complete and sign this form.  
The signed forms (from all members) are to be appended to the final report for submission.

### STAR GYM MEMBERSHIP SYSTEM

by

**1E\_GROUP 3**

**Group Member's Name** : Dennis Chew Yu Heng

**Group Member's ID** : 1231303567

For the purpose of completing this project, I have performed the following tasks:

- Coding for payment system
- Bug testing and modification
- State all the Implementation Details
- Wrote the conclusion
- Create Gantt chart for the project and check the report formatting

1. I hereby declare that I have assessed the final submission and I take the full responsibility should there be any incompleteness, omissions, delays, or non-submission.

2. I also certify that not part of this project is copied from any other student's work, or from any other sources, except where acknowledgement is made.

**Group Member's Signature** : *dennis*

**Group Member's Name** : Dennis Chew Yu Heng

**Group Member's ID** : 1231303567

**Date** : 20/1/2025

### Task Declaration Form

Each group member, including the group leader, must individually complete and sign this form.  
The signed forms (from all members) are to be appended to the final report for submission.

#### STAR GYM MEMBERSHIP SYSTEM

by

**1E\_GROUP 3**

**Group Member's Name** : Wong Chze Yee

**Group Member's ID** : 1231303074

For the purpose of completing this project, I have performed the following tasks:

- Involved in coding for milestone page
- Bug testing and modification
- Completed program screenshots with others member
- Cooperate with my teammate to complete the task

1. I hereby declare that I have assessed the final submission and I take the full responsibility should there be any incompleteness, omissions, delays, or non-submission.

2. I also certify that not part of this project is copied from any other student's work, or from any other sources, except where acknowledgement is made.

**Group Member's Signature**

:

*Wong*

**Group Member's Name**

:

Wong Chze Yee

**Group Member's ID**

:

1231303074

**Date**

:

22/1/25